Study Tracker 2.0 아키텍처 리디자인

1. Study Tracker 2.0 이 필요한 이유

1.0 은 단일 Spring Boot 애플리케이션 구조로 설계되었다.
Slack 이벤트 수신 → DB 저장 → 스케줄러 기반 요약 생성 → Slack 전송의 순차적 흐름이었다.
소규모 환경에서는 안정적으로 동작했지만, EC2 환경에서 운영 및 부하 테스트를 수행하며 구조적 한계를 확인했다.

① 배포 구조의 한계

- 애플리케이션 수정 시 Docker 이미지 재빌드 및 컨테이너 재기동 필요
- 배포 과정이 곧 서비스 중단으로 이어지는 구조

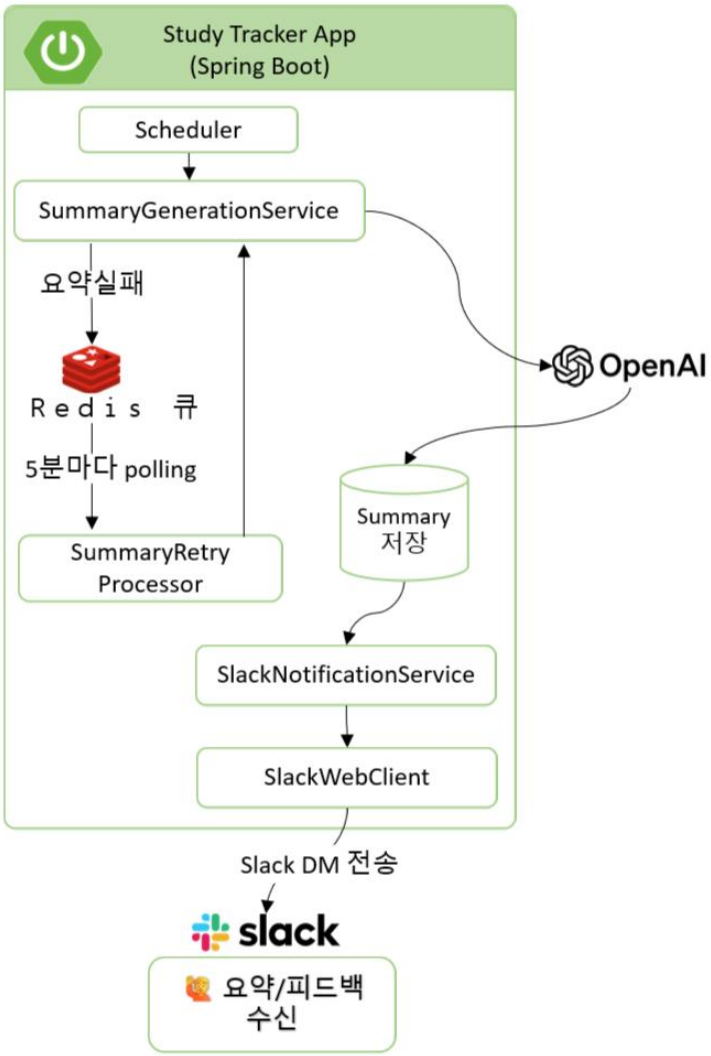
② 동기 처리 구조로 인한 병목

- 스케줄러 중심 순차 처리 구조
- OpenAI 호출 지연이 전체 흐름 지연으로 확장
- CPU 사용률 급증 및 트리거 지연 현상 발생

③ 책임 결합으로 인한 확장성 제한

- 요약 생성, Slack 전송, 실패 처리 로직이 하나의 흐름에 결합
- 기능 추가 시 기존 구조 변경 범위가 큼
- 장애 발생 시 전파 가능성 존재

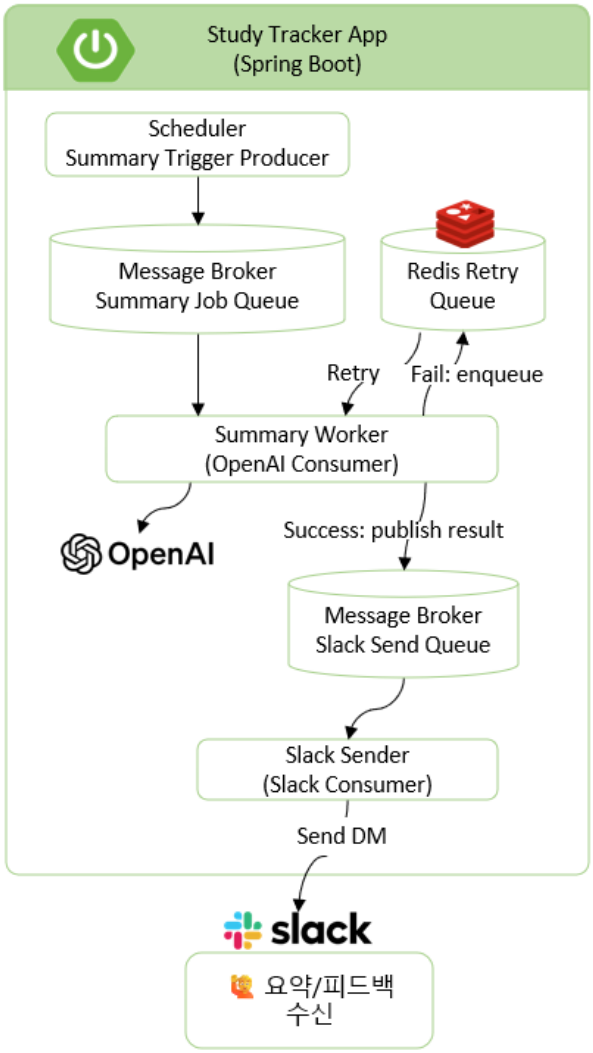
이 경험을 통해 2.0 은 기능 확장이 아니라,
비동기 전환과 책임 분리를 중심으로 한 구조 개선을 목표로 설계되었다.



2. 2.0 아키텍처 제안

2.0 은 기존 스케줄러 중심 순차 처리 구조를
작업 단위 이벤트 기반 구조로 재설계한 것이다.

1.0 에서는 스케줄러가 사용자 조회, OpenAI 호출, Slack 전송을 모두 수행했다.
2.0 에서는 각 책임을 분리하여 Producer–Consumer 구조로 전환했다.



구조 변경 핵심

- **Scheduler → Trigger 전용 Producer**
  - 작업을 직접 수행하지 않고 Summary Job Queue 에 이벤트만 발행
- **Summary Worker → OpenAI 처리 전담**
  - 요약 생성 및 결과 저장만 수행
  - 성공 시 Slack Send Queue 에 이벤트 발행

    - 실패 시 Retry Queue 로 위임
- **Slack Sender → 전송 전담 Consumer**
    - Slack API 호출 책임을 분리
    - Slack 장애가 요약 생성 흐름에 영향을 주지 않도록 격리

기대 효과

- OpenAI 지연이 Slack 전송에 영향을 주지 않음
- 작업 단위 병렬 확장 가능
- 장애 전파 최소화
- 책임 분리로 기능 추가 범위 축소

3. 1.0 vs 2.0 비교

구분	1.0	2.0
처리 구조	스케줄러 중심 순차 처리	이벤트 기반 비동기 처리
책임 분리	요약/전송 결합	Worker / Sender 분리
메시징	없음	Message Broker 기반 설계
재시도	Redis 큐	Worker 단위 재시도
DB	H2	PostgreSQL
배포	수동 Docker 재기동	CI 기반 자동화 목표

4. 기술 선택 근거

메시지 브로커

메시지 브로커(Kafka 또는 Redis Stream)를 고려한 이유는 다음과 같다.

- Producer–Consumer 구조 분리
- 작업 단위 병렬 처리 확장성 확보
- 장애 격리 및 처리 지연 완충
- 향후 다중 인스턴스 확장 가능성 대비

현재 규모에서는 즉시 도입 시 오버엔지니어링이 될 수 있어, 구조적 방향성 정리 수준으로 설계하였다.

데이터베이스 전환 (H2 → PostgreSQL)

1.0 은 개발 편의성을 위해 H2 를 사용했다.
2.0 에서는 비동기 처리 구조와 동시성 검증을 고려해 PostgreSQL 전환을 설계했다.

- MVCC 기반 안정적인 동시성 처리
- 실제 운영 환경과의 일관성 확보

DB 변경은 기능 확장이 아니라,
확장 가능한 구조 검증을 위한 선택이다.

5. CI/CD 관점 개선

1.0 은 Docker Compose 기반 수동 배포 구조였다.
애플리케이션 수정 시 컨테이너 재기동이 필요했고, 이는 서비스 영향으로 이어졌다.

2.0 에서는:

- GitHub Actions 기반 자동 빌드/배포

- Docker 이미지 버전 관리 자동화
- 재기동 최소화를 고려한 배포 흐름 설계

를 통해 운영 안정성을 개선한다.

6. Trade-off 및 설계 인사이트

2.0 설계는 확장성과 책임 분리를 고려한 개선안이지만,
현재 프로젝트 규모에서는 오버엔지니어링이 될 수 있다.

이 경험을 통해 다음을 인식했다.

- 모든 문제에 메시지 브로커가 정답은 아니다.
- 구조 복잡도는 팀 규모와 트래픽 수준에 맞춰 결정되어야 한다.
- 설계는 “기술 적용”이 아니라 “문제 해결”을 기준으로 선택해야 한다.

본 설계는 구조적 방향성을 정리한 것이며,
실제 적용은 단계적으로 이루어져야 한다.